

PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR ## PUCE --- ### INFORME TÉCNICO FINAL —
RETO 3 ### Sistema de E-Commerce de Vuelos ##### Arquitectura de Microservicios, API Gateway,
Event Bus, Web y Móvil --- **Asignatura:** Integración de Sistemas **Autor:** Juan Cevallos
Correo: juanccevallos12@gmail.com **Fecha:** 23 de junio de 2026 ---

Tabla de contenido

1. Introducción y objetivo
2. Alcance del Reto 3
3. Arquitectura final del sistema
4. Componentes y microservicios
5. Stack tecnológico
6. Estructura del repositorio
7. API Gateway e interoperabilidad (REST · GraphQL · gRPC)
8. Event Bus con RabbitMQ
9. Frontend web — Angular (panel administrativo)
10. Aplicación móvil — Flutter (marketplace)
11. Seguridad
12. Integración y gestión de datos (Prisma por dominio · ETL)
13. Observabilidad y trazabilidad
14. Contratos versionados (API v2)
15. Pruebas y verificación
16. Despliegue (Docker · Render)
17. Incidencias resueltas
18. Checklist de defensa
19. Riesgos y trabajo futuro
20. Conclusiones

1. Introducción y objetivo

El presente informe documenta la solución final (Reto 3) del **Sistema de E-Commerce de Vuelos**, desarrollado para la asignatura de Integración de Sistemas de la Pontificia Universidad Católica del Ecuador (PUCE).

El Reto 3 consolida la solución completa de integración: **microservicios, API Gateway, Event Bus, aplicación móvil, frontend web administrativo, seguridad, observabilidad, resiliencia y pruebas**. La arquitectura evoluciona desde el backend monolítico inicial (Reto 1) hacia una solución híbrida orientada a dominios, demostrando interoperabilidad entre distintos estilos de integración (REST, GraphQL, gRPC y mensajería asíncrona).

2. Alcance del Reto 3

- Descomposición del backend en **seis microservicios** por dominio.
- **API Gateway** unificado con REST público (/api/v2), agregador **GraphQL** y propagación de identidad y trazabilidad.
- **Event Bus con RabbitMQ** (exchange principal, retry y DLQ) reemplazando el bus in-process, con fallback local.
- **App móvil Flutter** tipo marketplace (búsqueda, reserva, pago, perfil, panel admin) y APK generado.
- **Frontend web Angular** con panel administrativo CRUD de todos los dominios.
- **Seguridad** con JWT, roles, ownership, API key interna, Helmet, CORS y rate limiting.
- **Separación de datos por dominio** con clientes Prisma independientes.
- **ETL/ELT operativo** que materializa un read model para auditoría.
- **Observabilidad** con OpenTelemetry, Jaeger, health checks y `x-correlation-id`.
- **Contratos versionados** (OpenAPI, GraphQL SDL, JSON Schema de eventos, `.proto`).
- **Despliegue** mediante `docker-compose` (local) y `render.yaml` (público v2).

3. Arquitectura final del sistema

flowchart LR

```
Mobile[Flutter marketplace movil] --> Gateway[API Gateway REST + GraphQL]
Web[Admin Angular] --> Gateway
```

```
Gateway --> Auth[auth-service]
Gateway --> Catalog[catalog-service]
Gateway --> Flights[flights-service REST + gRPC]
Gateway --> Booking[booking-service]
Gateway --> Payments[payments-service]
Gateway --> Admin[admin-service]
```

```
Booking --> Rabbit[(RabbitMQ Event Bus)]
Payments --> Rabbit
Admin --> Rabbit
```

```
Auth --> DBAuth[(identity schema)]
Catalog --> DBCatalog[(catalog schema)]
Flights --> DBFlights[(catalog/booking read schema)]
Booking --> DBBooking[(booking/checkin schema)]
Payments --> DBPayments[(payments schema)]
Admin --> DBAudit[(audit schema)]

Gateway --> Jaeger[Jaeger / tracing]
```

El sistema se organiza en tres planos:

- **Plano de clientes:** app móvil Flutter y web Angular.
- **Plano de integración:** API Gateway (REST + GraphQL) y Event Bus RabbitMQ.
- **Plano de dominio:** seis microservicios con bases de datos separadas por esquema lógico.

4. Componentes y microservicios

Componente	Responsabilidad
api-gateway	Entrada REST y agregador GraphQL. Propaga JWT y <code>x-correlation-id</code> .
auth-service	Autenticación, registro, perfil y verificación interna de tokens.
catalog-service	Países, ciudades, aeropuertos, aerolíneas, aeronaves y servicios.
flights-service	Búsqueda de vuelos y API gRPC para consultas internas.
booking-service	Reservas, pasajeros, asientos y pases de abordar.
payments-service	Pagos, facturación y servicios adicionales.
admin-service	Panel administrativo como orquestador de dominios.
rabbitmq	Event Bus con exchange principal, retry y DLQ.
vuelos-mobile	App Flutter para búsqueda, reserva, detalle y pago.
vuelos-angular	Web Angular para panel administrativo y operación.

5. Stack tecnológico

Capa	Tecnologías
Backend / microservicios	Node.js 20, TypeScript 5.9, Express 5, Zod (validación)
ORM / datos	Prisma 5.22 (cliente por dominio), PostgreSQL
API Gateway	Express 5, <code>http-proxy-middleware</code> , GraphQL 16, GraphQL Yoga 5, DataLoader

Capa	Tecnologías
Comunicación interna	gRPC (@grpc/grpc-js , @grpc/proto-loader)
Mensajería / eventos	RabbitMQ 3.13 (management), amqplib
Seguridad	jsonwebtoken (JWT), bcryptjs , Helmet, CORS, express-rate-limit
Observabilidad	OpenTelemetry SDK, exporter OTLP HTTP, Jaeger 1.57
Documentación de API	Swagger (swagger-jsdoc , swagger-ui-express), OpenAPI
Frontend web	Angular 21, RxJS 7.8, TailwindCSS 3.4, date-fns, Vitest
Frontend móvil	Flutter (Dart SDK 3.11), paquete http
Contenerización / despliegue	Docker, docker-compose, Render (render.yaml)
Pruebas	Node test runner (node --test), flutter test , Vitest

6. Estructura del repositorio

```

Sistema_vuelos.v1/
├── api-gateway/           # API Gateway REST + GraphQL (Node/TS)
│   └── src/
│       ├── config/registry.ts    # registro de upstreams
│       ├── proxy/router.ts       # proxy /api/v2 -> /api/v1
│       ├── graphql/             # schema, resolvers, dataloaders
│       ├── middleware/          # auth, CORS, rate limit, correlation
│       └── telemetry/           # OpenTelemetry
├── vuelos-backend/        # Microservicios de dominio (Node/TS)
│   ├── src/services/          # auth, catalog, flights, booking, payments, admin
│   ├── src/grpc/proto/        # contratos gRPC (.proto)
│   ├── src/scripts/           # ETL, validación de entorno, cuentas de servicio
│   └── prisma/services/        # un schema Prisma por dominio
├── vuelos-angular/        # Frontend web (Angular 21)
│   └── src/app/
│       ├── core/services/      # 11 servicios HTTP de dominio
│       ├── pages/              # auth, flights, reservations, admin (CRUD)
│       └── layouts/             # main-layout, admin-layout
├── vuelos-mobile/         # App móvil (Flutter)
│   └── lib/                   # main.dart, admin/admin_panel.dart
├── contracts/              # contratos versionados v2 (REST, GraphQL, eventos)
├── docs/                   # documentación técnica
├── docker-compose.yml      # orquestación local
└── render.yaml             # blueprint de despliegue público v2

```

7. API Gateway e interoperabilidad (REST · GraphQL · gRPC)

El gateway expone la segunda versión pública bajo `/api/v2/*`, mantiene `/api/v1/*` como compatibilidad y agrega **GraphQL** en `/graphql` para consultas compuestas. Esto demuestra interoperabilidad entre estilos:

- **REST público** para frontend web, móvil y pruebas.
- **GraphQL aggregator** para reducir el número de llamadas del cliente (usa DataLoader para evitar el problema N+1).
- **gRPC** en `flights-service` para integraciones internas de baja latencia.
- **Eventos asíncronos** por RabbitMQ para desacoplar booking, payments y auditoría.

El proxy traduce `/api/v2/*` hacia los upstream `/api/v1/*` de cada microservicio y agrega la cabecera `x-api-version: 2`. Correcciones aplicadas en Reto 3:

- `myReservations` usa `/api/v1/reservations/my`.
- `assignSeat` resuelve el pasajero y usa la ruta real de asignación de asiento.
- Los pagos de una reserva usan `/api/v1/payments/by-reservation/:reservationId`.
- Los boarding passes se agregan por pasajero con `/api/v1/boarding-passes/by-passenger/:passengerId`.
- `ServiceClient` tiene timeout y propaga `x-correlation-id`.

Contratos gRPC definidos (`vuelos-backend/src/grpc/proto/`): `flights.proto`, `reservations.proto`, `passengers.proto`, `payments.proto`, `checkin.proto`, `promotions.proto`.

8. Event Bus con RabbitMQ

RabbitMQ reemplaza el bus in-process como mecanismo principal de eventos, manteniendo fallback local si el broker no está disponible (para desarrollo).

Eventos publicados: `reservation.created`, `reservation.cancelled`, `seat.assigned`, `checkin.completed`, `payment.registered`, `invoice.issued`, `ancillary.added`.

Topología:

Elemento	Nombre
Exchange principal	<code>vuelos.events</code>
Exchange de retry	<code>vuelos.events.retry</code>
Exchange DLQ	<code>vuelos.events.dlq</code>
Cola por servicio	<code>\${SERVICE_NAME}.events</code>
Cola retry por servicio	<code>\${SERVICE_NAME}.events.retry</code>
Cola DLQ por servicio	<code>\${SERVICE_NAME}.events.dlq</code>

Resiliencia: mensajes persistentes, retry con TTL configurable, DLQ al superar `RABBITMQ_MAX_RETRIES`, idempotencia en memoria por `eventId`, pagos idempotentes por `transactionId` y `reservationId`,

y fallback in-process.

```
EVENT_BUS_DRIVER=rabbitmq
RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
RABBITMQ_EXCHANGE=vuelos.events
RABBITMQ_RETRY_EXCHANGE=vuelos.events.retry
RABBITMQ_DLQ_EXCHANGE=vuelos.events.dlq
RABBITMQ_RETRY_DELAY_MS=5000
RABBITMQ_MAX_RETRIES=3
EVENT_BUS_IDEMPOTENCY_WINDOW=1000
```

9. Frontend web — Angular (panel administrativo)

La web `vuelos-angular` (Angular 21 + TailwindCSS) implementa el panel administrativo y la operación de usuario. Estructura por capas:

- `core/services/` — 11 servicios HTTP de dominio: `auth`, `flights`, `reservations`, `payments`, `invoices`, `promotions`, `passenger-services`, `boarding-passes`, `airports`, `airline-service-configs`, `admin`.
- `core/` — guards (rutas protegidas), interceptors (JWT), models y store.
- `pages/` — `auth` (login, register), `flights` (home, results, detail), `reservations` (my-trips, reservation, detail) y `admin` con CRUD completo de 23 entidades: países, ciudades, aeropuertos, aerolíneas, aeronaves, vuelos, segmentos, clases de vuelo, reservas, pasajeros de reserva, pagos, facturas, ítems de factura, perfiles de facturación, pases de abordar, servicios, servicios de pasajero, configuraciones de servicios por aerolínea, promociones, usuarios, auditoría y dashboard.
- `layouts/` — `main-layout` (cliente) y `admin-layout` (administración).

Nota operativa: durante el Reto 3 la web fue repuntada del API Gateway v2 (que quedó fuera de servicio) al backend operativo <https://integracion-sistemas2026.onrender.com/api>. Detalle en la sección 17.

10. Aplicación móvil — Flutter (marketplace)

La app `vuelos-mobile` cumple el entregable de frontend móvil tipo marketplace.

Funciones implementadas:

- Búsqueda de vuelos por origen, destino, fecha, pasajeros y clase.
- Reserva de vuelo con datos de pasajero.
- Registro de usuario, inicio y cierre de sesión.
- Listado de reservas del usuario y detalle con pasajeros y pagos.
- Cancelación de reserva desde la app.
- Pago de reserva por `VISA`, `MASTERCARD`, `PAYPAL` o `TRANSFER`.

- Sección de perfil con datos reales y edición de campos.
- **Panel admin** (CRUD genérico) visible solo para rol `ADMIN`.
- Configuración de API por `--dart-define=API_BASE_URL=...`.

Configuración del backend: la app apunta por defecto a `https://integracion-sistemas2026.onrender.com/api` (timeout de 25 s).

APK generado: `vuelos-app-admin-fix.apk` (raíz del repositorio) y `vuelos-mobile/build/app/outputs/flutter-apk/app-release.apk`.

Comandos verificados: `flutter analyze`, `flutter test`, `flutter build apk`.

11. Seguridad

- **JWT obligatorio** en rutas de usuario.
- **Roles:** `CUSTOMER`, `ADMIN` y `SERVICE`.
- Middleware `requireAdmin` para administración.
- Middleware de **ownership** para reservas, pasajeros, pagos, facturas y perfiles.
- `INTERNAL_API_KEY` para endpoints internos entre servicios.
- **Helmet** y **CORS** en el gateway.
- **Rate limiting** global, en login/register y en búsqueda.
- Propagación de `x-correlation-id` para trazabilidad de extremo a extremo.
- Contraseñas con hash `bcryptjs`.

12. Integración y gestión de datos (Prisma por dominio · ETL)

La base se separa por dominios lógicos, cada uno con su propio schema Prisma:

Dominio	Contenido
<code>identity</code> (auth)	Usuarios y credenciales
<code>catalog</code>	Catálogos y vuelos
<code>flights</code>	Lectura de catálogo/booking para búsqueda
<code>booking</code>	Reservas, pasajeros y check-in
<code>payments</code>	Pagos, facturas y perfiles de facturación
<code>admin</code> (audit)	Logs de auditoría

El `admin-service` actúa como orquestador administrativo usando **clientes Prisma por dominio**, evitando que un microservicio use modelos que no existen en su schema.

ETL/ELT operativo


```
cd vuelos-backend
npm run etl:reto3
```

El script `src/scripts/reto3-etl-report.ts` realiza:

- **Extract:** lee métricas de `identity`, `catalog`, `flights`, `booking`, `payments` y `audit`.
- **Transform:** conteos por dominio, estados, conversión de reservas pagadas, ingreso promedio por pasajero y reglas básicas de calidad de datos.
- **Load:** materializa el reporte en `docs/evidence/reto3-operational-report.json`, que funciona como read model operacional para defensa y auditoría.

13. Observabilidad y trazabilidad

- Health check del gateway con estado de los microservicios.
- Logs estructurados de eventos con `eventId`, `eventType`, `producer` y `cid`.
- **OpenTelemetry** instrumentado en cada servicio, exportando vía OTLP.
- **Jaeger 1.57** configurado en `docker-compose.yml` para tracing distribuido.
- `x-correlation-id` desde el gateway hasta microservicios y Event Bus.
- Estado de **circuit breakers** expuesto por el gateway.

14. Contratos versionados (API v2)

Artefactos de contrato versionados en `contracts/`:

- **REST / OpenAPI:** `contracts/rest/booking-api-v2.openapi.yaml`.
- **GraphQL SDL:** `contracts/graphql/schema-v2.graphql`.
- **Eventos RabbitMQ:** `contracts/events/domain-events-v2.schema.json`.
- **gRPC:** `vuelos-backend/src/grpc/proto/*.proto`.

El gateway publica los contratos desplegados:

- `GET /api/v2/contracts`
- `GET /contracts/rest/booking-api-v2.openapi.yaml`
- `GET /contracts/graphql/schema-v2.graphql`
- `GET /contracts/events/domain-events-v2.schema.json`

El contrato público de integración Booking v2 está documentado en `docs/contrato-api-booking-v2.md`.

15. Pruebas y verificación

```
# Backend
cd vuelos-backend
npm run build
npm test
```

```
# Gateway
cd ../api-gateway
npm run build
npm test
npm run smoke:v2      # smoke test contra v2 (local o GATEWAY_BASE_URL)

# Móvil
cd ../vuelos-mobile
flutter analyze
flutter test
flutter build apk
```

Validación de configuración de entorno:

```
cd vuelos-backend
npm run validate:reto3-env
```

El comando lista variables faltantes para una demo/despliegue completo: secretos JWT, API key interna, URLs de base por dominio, RabbitMQ externo y frontend.

16. Despliegue (Docker · Render)

Local (docker-compose): define `jaeger`, `api-gateway`, `auth-service`, `catalog-service`, `flights-service`, `booking-service`, `payments-service`, `admin-service` y `rabbitmq` (con management UI en `http://localhost:15672`), todos en la red `vuelos-net`.

Público v2 (`render.yaml`): blueprint raíz que publica la versión 2 como servicios Node independientes:

- `vuelos-api-gateway-v2`
- `vuelos-auth-service-v2`
- `vuelos-catalog-service-v2`
- `vuelos-flights-service-v2`
- `vuelos-booking-service-v2`
- `vuelos-payments-service-v2`
- `vuelos-admin-service-v2`

RabbitMQ se configura como broker externo administrado (p. ej. CloudAMQP) vía `RABBITMQ_URL`; Jaeger/OTLP mediante `OTEL_EXPORTER_OTLP_ENDPOINT`.

17. Incidencias resueltas

Web sin reflejar datos (API mal apuntada). La web Angular apuntaba al API Gateway v2 (`vuelos-api-gateway-v2.onrender.com`), que quedó fuera de servicio y devolvía **404 en todas las rutas**, por lo

que ningún listado, login ni panel mostraba contenido. La app móvil ya había sido migrada al backend operativo (`integracion-sistemas2026.onrender.com/api`).

Solución: se repuntaron las **12 referencias** de URL de la web al backend operativo. Verificación posterior: las rutas (`/auth/login` , `/auth/me` , `/flights` , `/flights/search`) responden correctamente y no quedan referencias al gateway caído. Detalle completo en `docs/informe-tecnico-fix-web-api.md` .

18. Checklist de defensa

- Mostrar `docker-compose.yml` con gateway, microservicios, RabbitMQ y Jaeger.
- Abrir RabbitMQ Management en `http://localhost:15672` .
- Ejecutar búsqueda desde Flutter; registrar usuario o iniciar sesión.
- Crear y pagar una reserva desde Flutter.
- Ver eventos `ReservationCreated` y `PaymentRegistered` en logs/RabbitMQ.
- Mostrar health del gateway y estado de circuit breakers.
- Mostrar trazabilidad por `x-correlation-id` .
- Explicar retry, DLQ e idempotencia del Event Bus.
- Explicar separación de schemas y clientes Prisma por dominio.
- Mostrar contratos REST/OpenAPI, GraphQL y de eventos.
- Ejecutar `npm run smoke:v2` en `api-gateway` .
- Mostrar `render.yaml` como evidencia de despliegue público v2.
- Ejecutar `npm run etl:reto3` y mostrar el reporte operacional generado.
- Mostrar el panel administrativo Angular operando contra el backend.

19. Riesgos y trabajo futuro

- La validación end-to-end depende de tener Docker disponible.
- El despliegue público requiere llenar variables secretas en Render o equivalente.
- La idempotencia actual del Event Bus es en memoria; para producción se recomienda persistir los `eventId` procesados por consumidor.
- Los retries de RabbitMQ usan TTL por cola; para políticas más avanzadas conviene configurar colas por intervalo o un plugin de delayed messages.
- **Centralizar la URL del backend** en la web (un único `environment`) para evitar desincronizaciones como la descrita en la sección 17.
- Definir el futuro del **API Gateway v2**: restaurarlo o retirarlo de contratos y documentación.

20. Conclusiones

El Reto 3 entrega una solución de integración completa y funcional que cumple los objetivos planteados: una arquitectura de microservicios orientada a dominios, integrada mediante un API Gateway que combina **REST, GraphQL y gRPC**, y un **Event Bus RabbitMQ** resiliente con retry, DLQ e idempotencia. La solución se complementa con dos clientes —una **app móvil Flutter** tipo marketplace y un **panel web Angular** administrativo—, controles de **seguridad** transversales,

observabilidad con OpenTelemetry y Jaeger, **contratos versionados** y mecanismos de **despliegue** tanto local (Docker) como público (Render).

El sistema demuestra de forma práctica los principios de integración de sistemas: desacoplamiento por mensajería, interoperabilidad entre estilos de comunicación, versionado de contratos, separación de datos por dominio y trazabilidad de extremo a extremo.

Pontificia Universidad Católica del Ecuador — PUCE *Integración de Sistemas · Reto 3 · Juan Cevallos · 2026*